TRAINOCATE

Module 3

Automation, monitoring and major incidents

One day • 390 instructional minutes

Investigate → explain → mitigate → verify → hand over

TRAINOCATE

Today’s route

60 min | Extend your Bash script into a health check
55 min | Automated evidence collection
40 min | Log parsing and threshold monitoring
20 min | Slow-query summaries and correlation pipelines
35 min | Health checks, cron and safe retention
20 min | Deployment verification and maintenance boundaries
110 min | Major incident simulation
30 min | Final challenges and operational handover
20 min | Checkpoints and recovery reserve

The lab system and its boundaries

LEARNER

SSH + individual account

Team workspace

APPLICATION HOST

Tomcat + MySQL

Loyalty training app

DATABASE LAB

SQL Server 2022

Query Store + Agent

Request IDs connect application evidence.

Synthetic packs cover replication, IIS, SELinux and pooled-app incidents.

Fault injection and storage administration are instructor-controlled.

How to work an incident

1. State impact and capture UTC time.
2. Preserve evidence before changing the system.
3. Test a hypothesis with a discriminating check.
4. Make one scoped change with a rollback.
5. Verify the affected business function and communicate.

TRAINOCATE

Extend your Bash script into a health check

Turn yesterday’s commands into a script another engineer can run reliably.

- Quote expansions; treat arguments as data; use arrays for command arguments.
- Exit codes are an interface: document healthy, unhealthy and usage outcomes.
- set -e is not a substitute for deliberate error handling.

Your investigation sequence

1. Create a script you can edit
2. Implement the check interface
3. Check syntax before execution
4. Test an expected failure without changing services
5. Test your own validation and array behavior
6. Compare with the reference implementation

Create a script you can edit

```
mkdir -p ~/support-lab/bin
cat "$HOME/support-lab/bin/triage.sh"
if [[ ! -e "$HOME/support-lab/bin/my-health.sh" ]]; then
    cp "$HOME/support-lab/bin/triage.sh" "$HOME/support-lab/bin/my-health.sh"
fi
nano "$HOME/support-lab/bin/my-health.sh"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Your script returns 0 for healthy checks, 1 for an unreachable endpoint and 2 for invalid arguments or missing tools.

✓ Explain the quoted option array; submit actual results from your own script.

? What happens to $? after echo runs?

? Why can set -e behave differently inside conditionals?

Debrief and deliverable

A documented executable Bash health checker and test transcript.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Automated evidence collection

An escalation needs a portable evidence bundle even when some commands fail or lack permission.

- Use restrictive permissions, unique timestamped directories and a manifest.
- Capture stdout, stderr and exit status for each check.
- Redaction needs review; a regex cannot guarantee removal of every secret.

Your investigation sequence

1. Collect a bounded evidence bundle
2. Inspect the exact bundle
3. Interpret missing permissions
4. Inspect and exercise masking with synthetic input
5. Write an evidence index
6. Transfer the method without copying

Collect a bounded evidence bundle

```
cd "$COURSE_ROOT"
shared/scripts/collect-evidence.sh "$HOME/support-lab/evidence/m3-02/bundles" '20 minutes ago'
collector_rc=$?
printf 'collector_exit=%s\n' "$collector_rc"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Failed checks remain visible in the manifest.

✓ Bundle contains no credentials or environment dumps and is reviewed before sharing.

? Can a timestamp alone prevent simultaneous filename collisions?

? Should permission denied be silently converted to an empty success file?

Debrief and deliverable

A reviewed evidence archive, manifest and evidence index.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

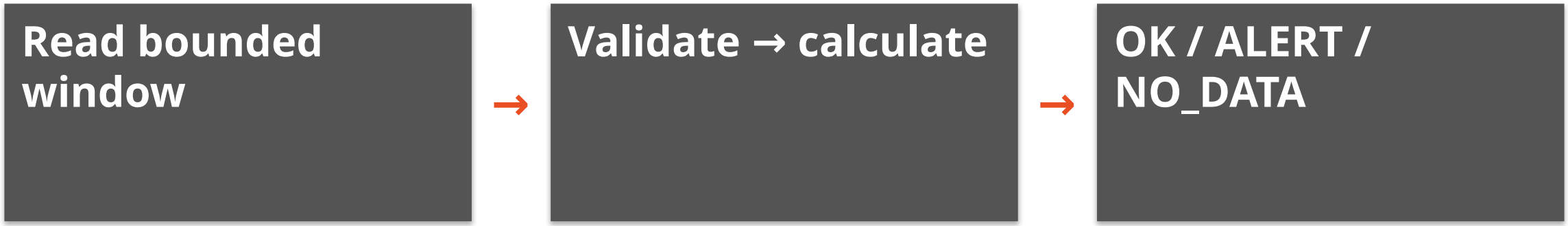
What remains uncertain?

Log parsing and threshold monitoring

Build a monitor that counts actual requests, rejects malformed rows and reports slow requests.

- Define the time window, denominator, threshold comparison and no-data behavior.
- Count status fields rather than words such as ERROR in free text.
- A monitoring result is only as useful as its handling of missing or stale input.

Monitoring must preserve unknown states



No valid records is unknown, not zero errors. A successful parse does not prove freshness.

Run the known fixture

```
cd "$COURSE_ROOT"
shared/scripts/monitor-log.sh shared/fixtures/application.log 10 1000
monitor_rc=$?
printf 'monitor_exit=%s\n' "$monitor_rc"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Fixture result is 6 requests, 2 errors, 33.33%, 2 slow, 1 malformed, ALERT.
- ✓ Empty input yields NO_DATA and exit 2, never healthy.

? Is zero observed errors equivalent to a healthy service when no requests exist?

? Should 404 responses count as server failures in this policy?

Debrief and deliverable

A monitoring CSV and a written policy with denominator and no-data semantics.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

Slow-query summaries and correlation pipelines

Create one summary combining application errors with database slow-query evidence without pretending they share a guaranteed join key.

- Use database-native summaries for multiline slow logs.
- Keep units, source window and provenance in exported results.
- A correlation hypothesis must be labeled if an exact join is unavailable.

Your investigation sequence

1. Create your report directory and source record
2. Produce two slow-query rankings
3. Save an alert result without losing it
4. Run the reference wrapper and inspect its manifest
5. Transfer the method without copying

Create your report directory and source record

```
mkdir -p ~/support-lab/summary
summary_dir=$(mktemp -d "$HOME/support-lab/summary/run-XXXXXX")
date -u +%FT%TZ > "$summary_dir/captured-at.txt"
printf '%s\n' "$summary_dir"
cd "$COURSE_ROOT"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Report preserves both frequency and time rankings.
- ✓ Wrapper does not discard useful ALERT output because its exit code is 1.

? Does one SQL statement always fit on one line?

? Can identical timestamps prove causal linkage?

Debrief and deliverable

A timestamped monitoring/slow-query report and its limitations.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Health checks, cron and safe retention

Your script works in a terminal but fails under cron. Make paths, permissions, concurrency and output explicit.

- Cron supplies a smaller environment and different working directory.
- Use absolute paths, a controlled PATH and a lock to prevent overlapping runs.
- Cleanup is dry-run first, narrowly scoped and separate from incident evidence collection.

Your investigation sequence

1. Install learner-owned scripts and test the cron environment
2. Back up and edit your own crontab
3. Verify the saved entry and observe execution
4. Create disposable retention input
5. Preview, apply and verify narrowly scoped cleanup
6. Remove only the exercise’s scheduled entries

Install learner-owned scripts and test the cron environment

```
mkdir -p ~/support-lab/{bin,logs,evidence}
cp "$COURSE_ROOT/shared/scripts/health-check.sh" ~/support-lab/bin/
cp "$COURSE_ROOT/shared/scripts/retention.sh" ~/support-lab/bin/
(cd /tmp && env -i HOME="$HOME" PATH=/usr/bin:/bin "$HOME/support-lab/bin/health-check.sh")
check_rc=$?
printf 'minimal_environment_exit=%s\n' "$check_rc"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Scheduled execution uses absolute paths and a per-user lock.

✓ Dry-run identifies only intended incident archives older than the retention boundary.

? Will a cron job inherit your interactive exported secrets?

? Why can overlapping checks create a monitoring problem?

Debrief and deliverable

A reviewed cron entry, minimal-environment test and retention dry-run output.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Deployment verification and maintenance boundaries

A deployment health gate must catch a business failure even if the process is active.

- Check service, readiness and one representative business response.
- Timeouts and nonzero exits must propagate to the caller.
- Automated diagnosis does not authorize automatic destructive repair.

Your investigation sequence

1. Record the business response
2. Create your deployment gate
3. Check and run the gate
4. Test an affected business function
5. Restore and verify
6. Transfer the method without copying

Record the business response

```
curl --fail --silent --max-time 5 'http://127.0.0.1:8080/loyalty/player?id=1'
curl --fail --silent --max-time 5 http://127.0.0.1:8080/loyalty/payment
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Gate fails on affected business functionality even when Tomcat remains active.
- ✓ Recovery checks cover the same function that originally failed.

? Can a generic /health route detect every business outage?

? Should the first failed probe trigger a database restart?

Debrief and deliverable

A deployment-check script with documented scope and failure behavior.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Major incident simulation

Run one of three scenarios: loyalty outage, delayed transactions, or payment integration failure. Rotate roles and deliver timed updates.

- Separate facts, hypotheses and decisions in the incident log.
- Assign owners and keep one coordinated change stream on the shared host.
- Mitigation reduces impact; RCA may remain provisional at handover.

Your investigation sequence

1. Prepare your submission workspace
2. Receive the incident and assign roles
3. Investigate within the time budget
4. Collect and inspect evidence
5. Propose one mitigation and define recovery
6. Verify and hand over

Prepare your submission workspace

```
cp -i "$COURSE_ROOT/shared/templates/incident-report.md"
"$HOME/support-lab/evidence/m3-07/incident.md"
date -u +%FT%TZ
nano "$HOME/support-lab/evidence/m3-07/incident.md"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Timeline links symptoms, actions and results to evidence.
- ✓ Updates communicate impact, current action, owner and next update time.

? Should you announce root cause when you only have correlation?

? Can unrelated background warnings distract from the affected business function?

Debrief and deliverable

A major incident report, evidence bundle and three stakeholder updates.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Final challenges and operational handover

Another shift must continue your work without a verbal briefing.

- A handover describes remaining work, evidence locations and safe next steps.
- RCA distinguishes trigger, root cause, contributing factors and detection gaps.
- Monitoring improvements need an owner and measurable acceptance criterion.

Your investigation sequence

1. Collect your handover inputs
2. Prepare a five-minute handover
3. Defend a counterexample
4. Write three actionable improvements
5. Review and submit
6. Transfer the method without copying

Collect your handover inputs

```
syntax_rc=0
for script in "$HOME"/support-lab/bin/*.sh; do
    [[ -f "$script" ]] || continue
    bash -n "$script" || syntax_rc=1
done
printf 'syntax_exit=%s\n' "$syntax_rc"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Next shift can reproduce checks and locate evidence.
- ✓ Final report distinguishes confirmed facts from probable explanations.

? Is “monitor more” an actionable recommendation?

? Can an RCA be useful when some uncertainty remains?

Debrief and deliverable

Operational handover and prioritized improvement backlog.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Advanced automation challenge bank

Harden the monitoring workflow against inconvenient inputs and execution contexts.

- Test adverse inputs, not only the happy path.
- Concurrency, stale data and time zones can create false confidence.
- Maintenance scripts need explicit boundaries and recovery plans.

Your investigation sequence

1. Create a hardening test matrix
2. Run the reference boundary checks
3. Implement and test your extensions
4. Design database automation boundaries
5. Review the final patch

Run the reference boundary checks

```
cd "$COURSE_ROOT"
shared/scripts/monitor-log.sh shared/fixtures/application.log 10 1000
first_rc=$?
shared/scripts/monitor-log.sh shared/fixtures/application.log 34 2000
second_rc=$?
printf 'first=%s second=%s\n' "$first_rc" "$second_rc"
shared/scripts/collect-evidence.sh "$HOME/support-lab/evidence/m3-09/output with spaces" '5 minutes ago'
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Each new behavior has a meaningful adverse-case test.
- ✓ Unknown/unavailable dependencies are not reported as healthy.

? Can a successful parser be monitoring yesterday’s data?

? What happens if a timeout exits 124 and the wrapper overwrites $?

Debrief and deliverable

A hardening patch, test transcript and documented remaining limitations.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Reference shelf and completion checks

Use shared/SOURCES.md for official technical references.

Submit scripts, evidence, interpretations and recovery checks.

Keep private keys and passwords out of submissions.

Complete the core labs before attempting extension challenges.

Evidence 25 • Diagnosis 25 • Recovery 20 • Automation 20 • Communication 10